LAPORAN PRAKTIKUM BASIS DATA
PERTEMUAN 1 : KONFIGURASI INDEXING PADA POSTGRESQL

Nama : Rasya Diecieafadhila
NIM : 2507053 – B
Kelas : Sistem Informasi
Kelompok : Alfisar & Rasya Dieciefadhila
Tanggal : 04 Oktober 2026

Tujuan Praktikum

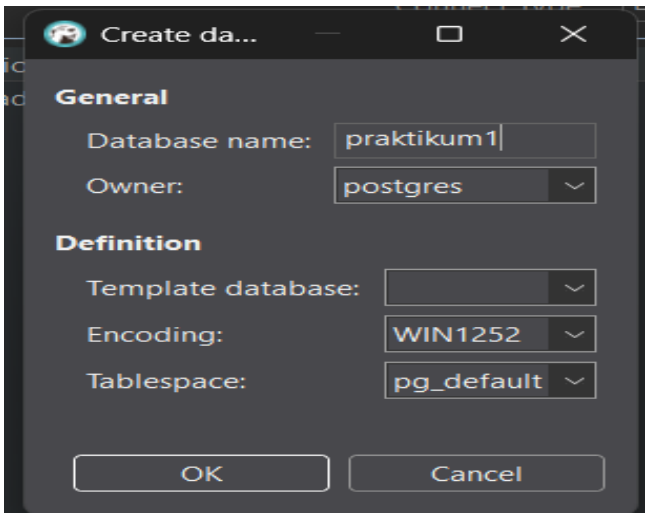
1. Menjelaskan konsep dan fungsi indexing pada basis data relasional.
2. Membuat basis data dan skema tabel relasional pada PostgreSQL.
3. Mempersiapkan dataset uji sederhana untuk kebutuhan pengujian praktikum.
4. Memverifikasi jumlah record dan karakteristik data.
5. Mengonfigurasi single-column index dan composite index.
6. Memeriksa dan memvalidasi struktur indeks yang telah dibuat melalui katalog system.
7. Memahami keuntungan, konsekuensi, dan biaya overhead dari penerapan indeks.

Lingkungan Praktikum

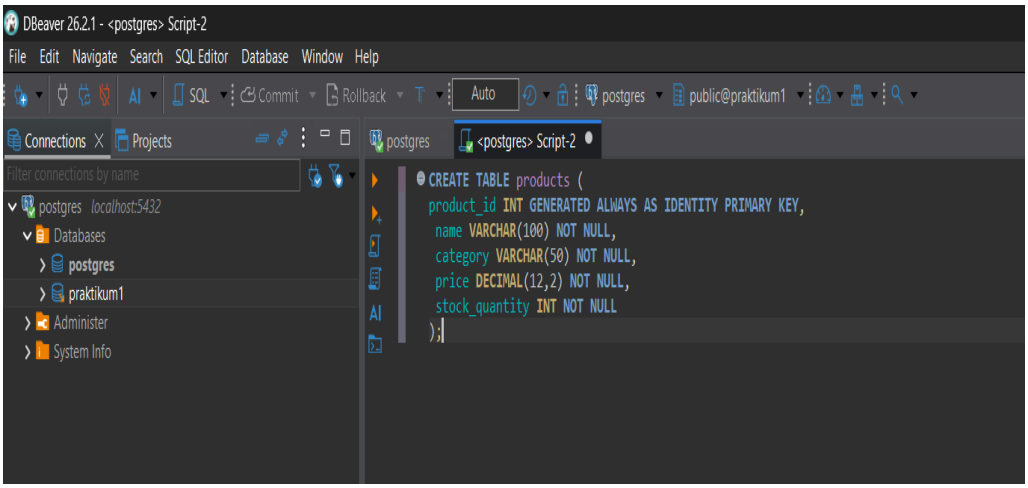
- **Sistem Operasi** : Windows 11
- **Database Server** : PostgreSQL 15/16
- **DBMS Client** : DBeaver Community Edition

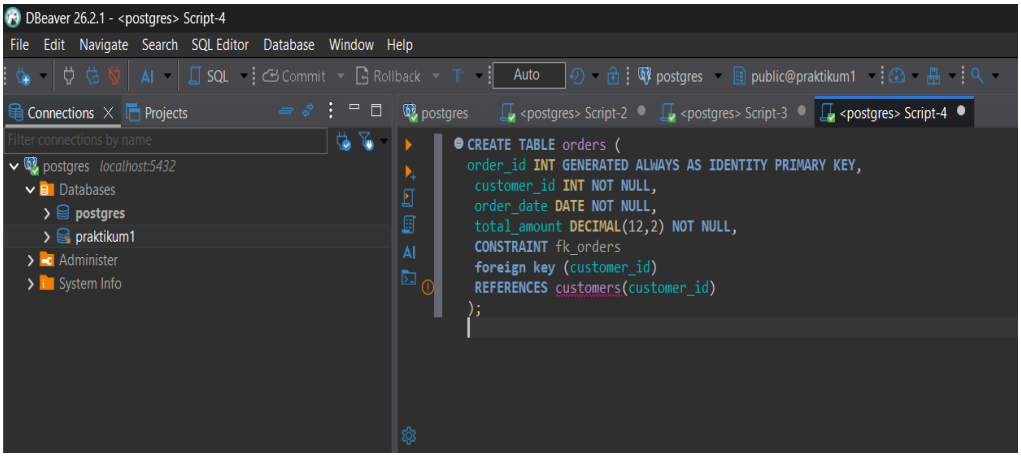
Dokumentasi & Hasil Pemeriksaan

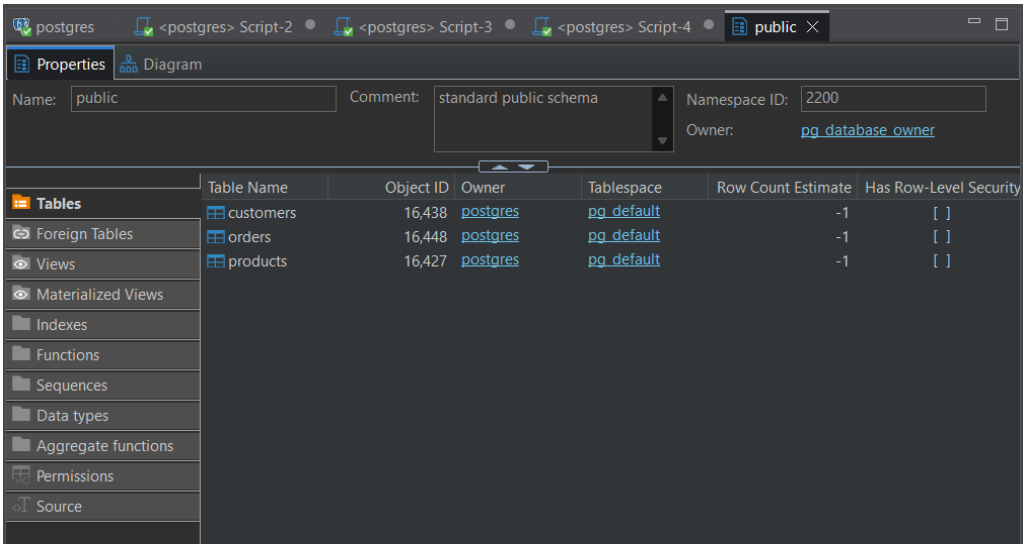
1. Buka DBngin, konekan ke postgresql lalu buat Database di DBngin dan create database baru di DBeaver, agar sesuai nama Praktikum, contoh praktikum1 lalu klik OK.



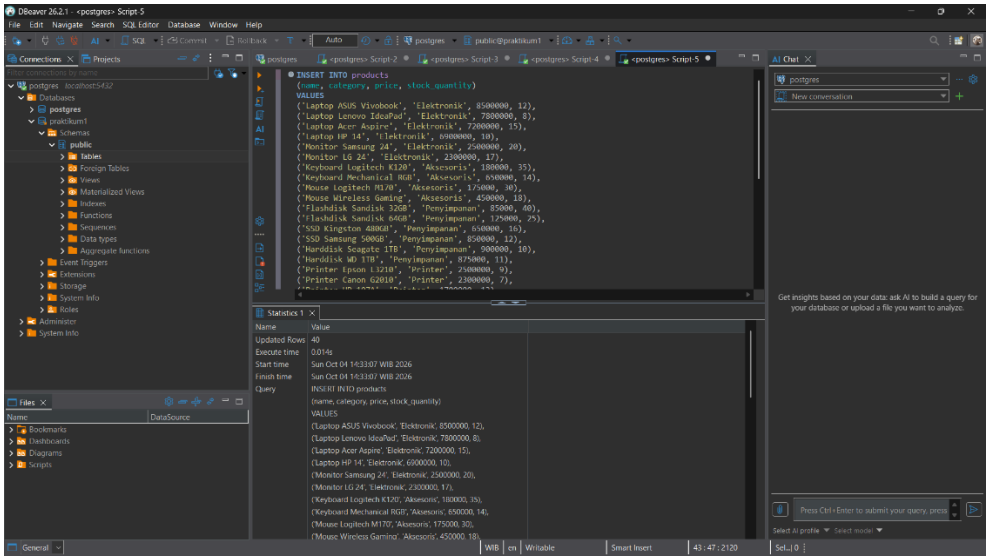
2. Lalu open SQL Editor dan masukkan querry untuk tahap pembuatan Table.

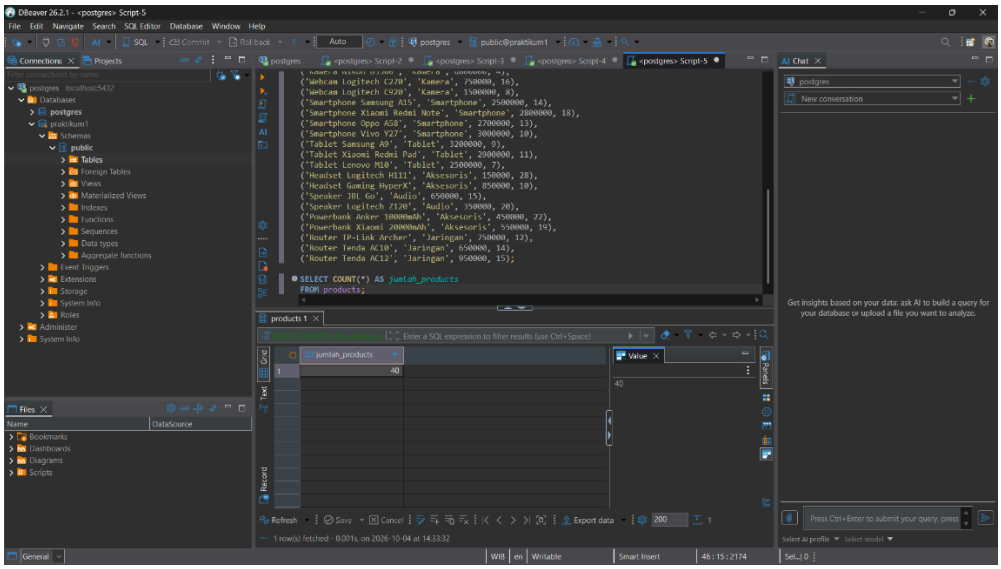


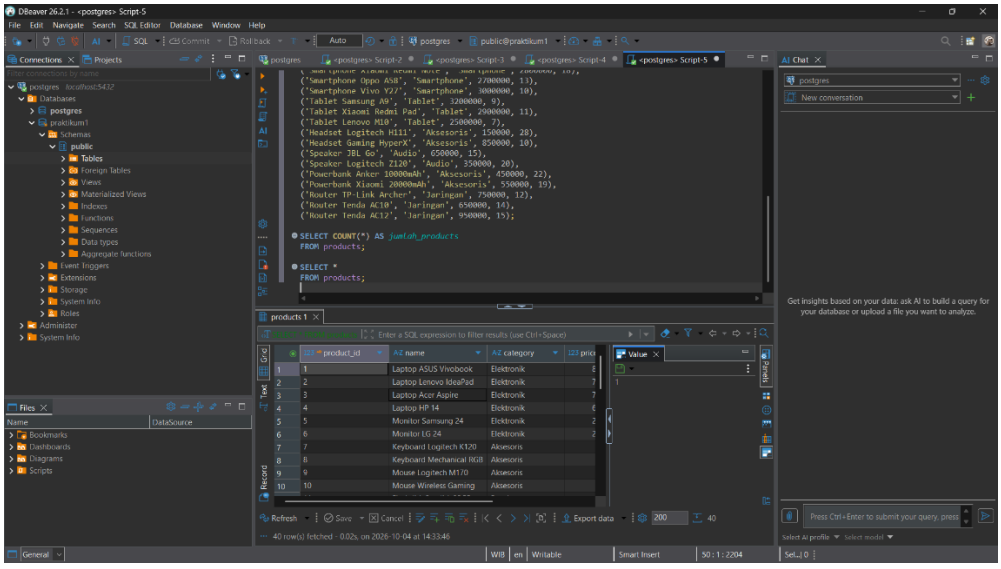


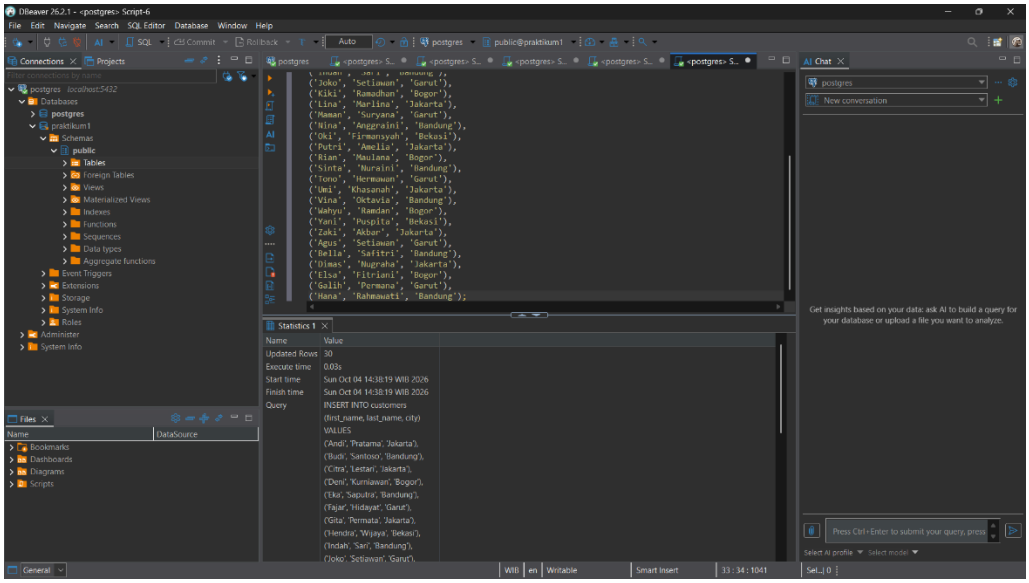


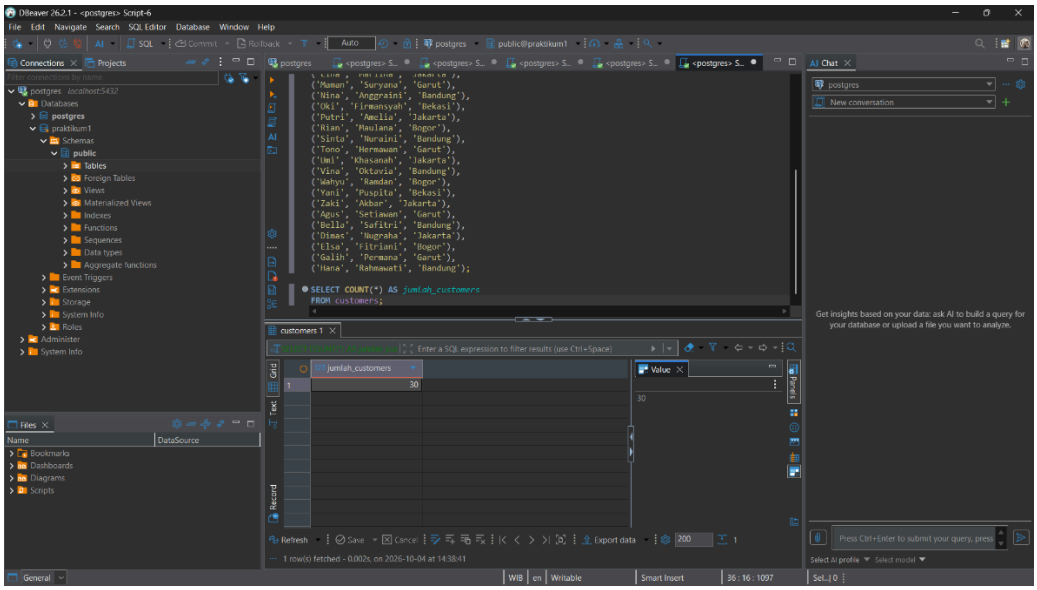
3. Lalu lanjut ke penginputan data set, 40 products, 30 costumers, dan 60 orders.

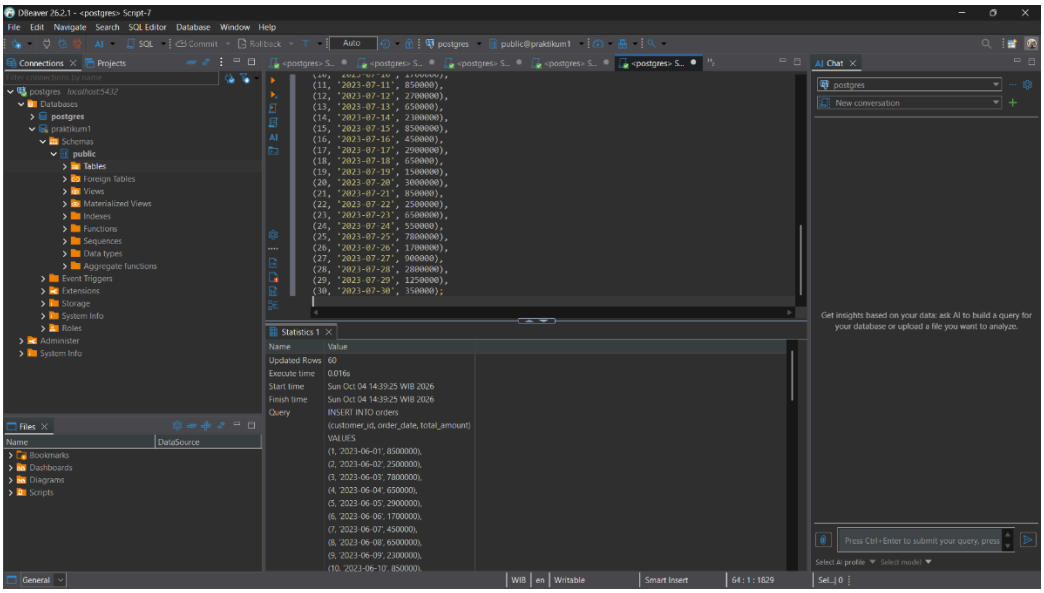


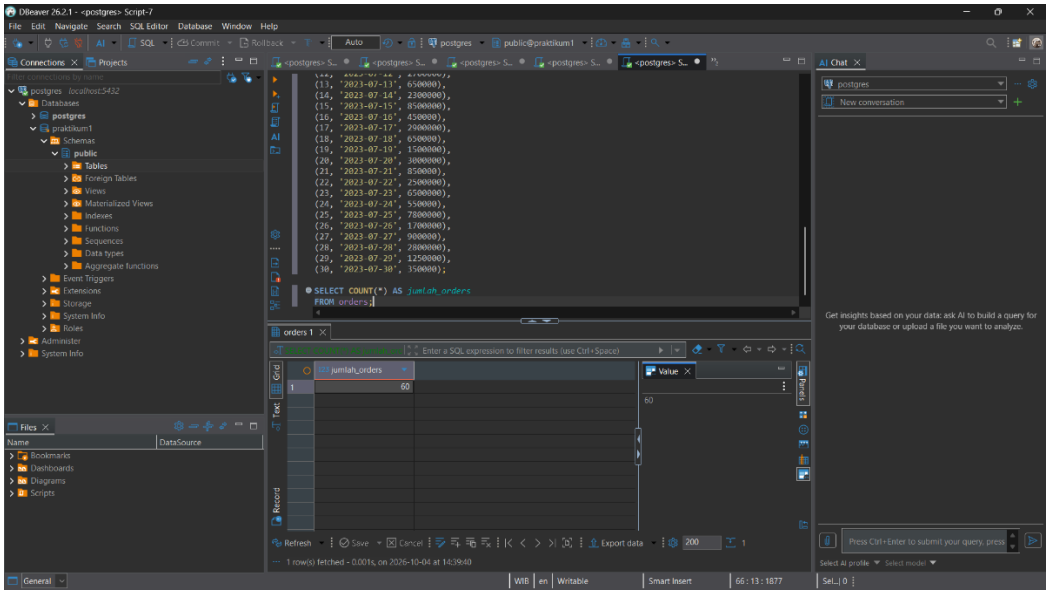


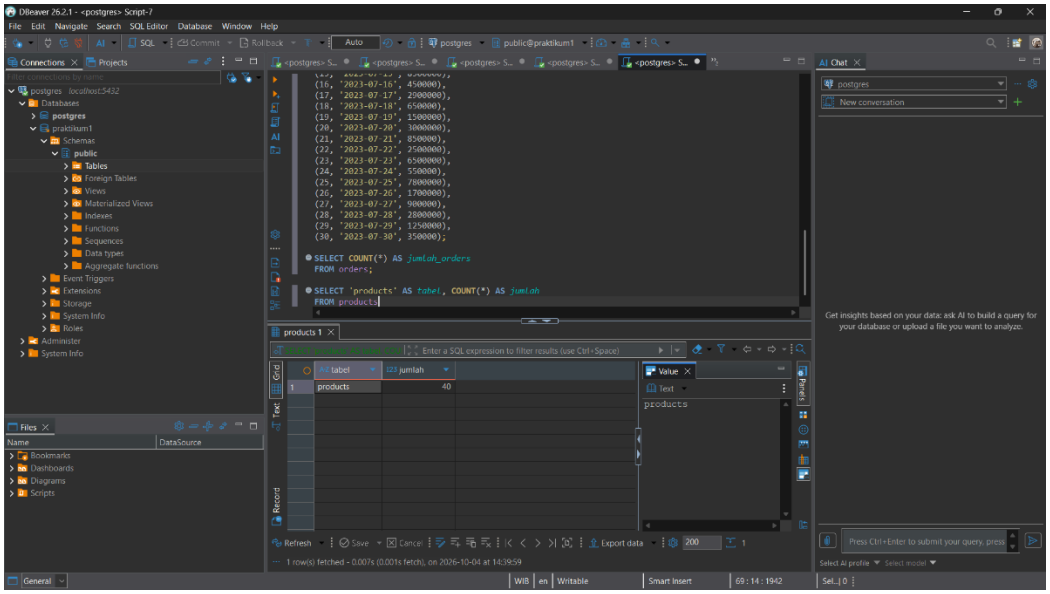


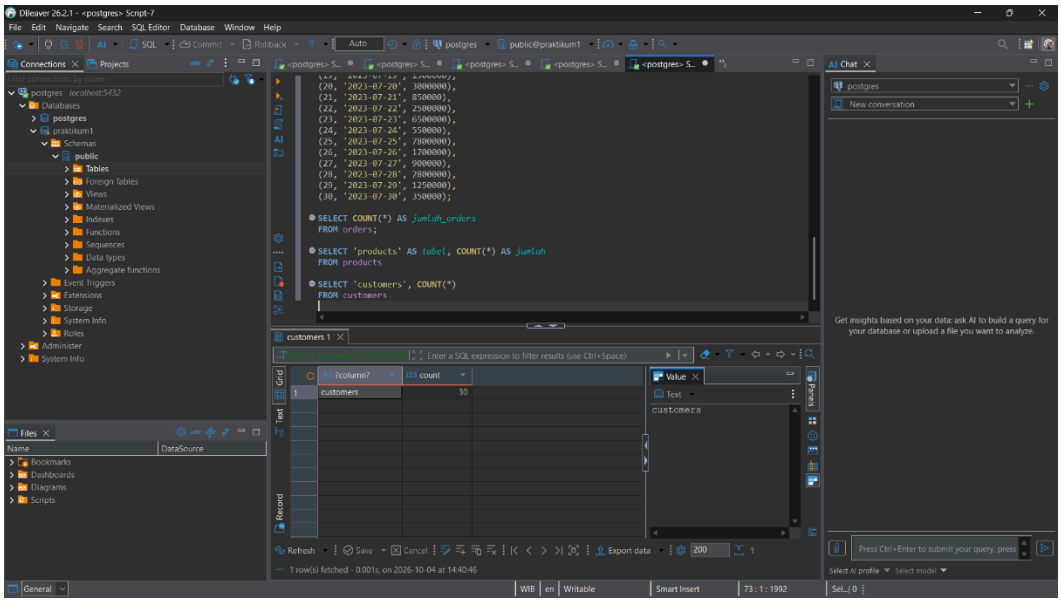

DBeaver 26.2.1 - <postgres> Script-6
('Joko', 'Setiawan', 'Garut'),
('Kiki', 'Ramadhan', 'Bogor'),
('Lina', 'Marlina', 'Jakarta'),
('Maman', 'Suryana', 'Garut'),
('Nina', 'Anggraini', 'Bandung'),
('Oki', 'Firmansyah', 'Bekasi'),
('Putri', 'Amelia', 'Jakarta'),
('Rian', 'Maulana', 'Bogor'),
('Sinta', 'Nuraini', 'Bandung'),
('Tono', 'Hermawan', 'Garut'),
('Umi', 'Khasanah', 'Jakarta'),
('Vina', 'Oktavia', 'Bandung'),
('Wahyu', 'Ramdan', 'Bogor'),
('Yani', 'Puspita', 'Bekasi'),
('Zaki', 'Akbar', 'Jakarta'),
('Agus', 'Setiawan', 'Garut'),
('Bella', 'Safitri', 'Bandung'),
('Dimas', 'Nugraha', 'Jakarta'),
('Elsa', 'Fitriani', 'Bogor'),
('Galih', 'Permana', 'Garut'),
('Hana', 'Rahmawati', 'Bandung');
Statistics 1
Name
Value
Updated Rows
30
Execute time
0.03s
Start time
Sun Oct 04 14:38:19 WIB 2026
Finish time
Sun Oct 04 14:38:19 WIB 2026
Query
INSERT INTO customers
(first_name, last_name, city)
VALUES
('Andi', 'Pratama', 'Jakarta'),
('Budi', 'Santoso', 'Bandung'),
('Citra', 'Lestari', 'Jakarta'),
('Deni', 'Kurniawan', 'Bogor'),
('Eka', 'Saputra', 'Bandung'),
('Fajar', 'Hidayat', 'Garut'),
('Gita', 'Permata', 'Jakarta'),
('Hendra', 'Wijaya', 'Bekasi'),
('Indah', 'Sari', 'Bandung'),
('Joko', 'Setiawan', 'Garut')
AI Chat
postgres
New conversation
Get insights based on your data: ask AI to build a query for your database or upload a file you want to analyze.
Press Ctrl+Enter to submit your query, press
Select AI profile
Select model
Files
Name
DataSource
Bookmarks
Dashboards
Diagrams
Scripts
General
WIB
en
Writable
Smart Insert
33 : 34 : 1041
Sel: 0

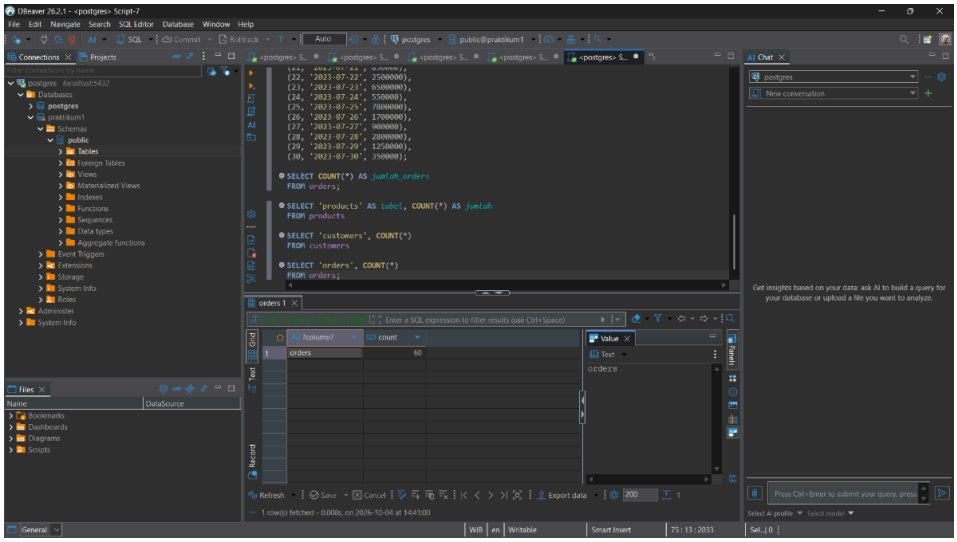

DBeaver 26.2.1 - <postgres> Script-6
('Maman', 'Suryana', 'Garut'),
('Nina', 'Anggraini', 'Bandung'),
('Oki', 'Firmansyah', 'Bekasi'),
('Putri', 'Amelia', 'Jakarta'),
('Rian', 'Maulana', 'Bogor'),
('Sinta', 'Nuraini', 'Bandung'),
('Tono', 'Hermawan', 'Garut'),
('Umi', 'Khasanah', 'Jakarta'),
('Vina', 'Oktavia', 'Bandung'),
('Wahyu', 'Ramdan', 'Bogor'),
('Yani', 'Puspita', 'Bekasi'),
('Zaki', 'Akbar', 'Jakarta'),
('Agus', 'Setiawan', 'Garut'),
('Bella', 'Safitri', 'Bandung'),
('Dimas', 'Nugraha', 'Jakarta'),
('Elsa', 'Fitriani', 'Bogor'),
('Galih', 'Permana', 'Garut'),
('Hana', 'Rahmawati', 'Bandung');
SELECT COUNT(*) AS jumlah_customers
FROM customers;
customers 1
jumlah_customers
Value
30
Refresh
Save
Cancel
Export data
1 row(s) fetched - 0.002s, on 2026-10-04 at 14:38:41
AI Chat
postgres
New conversation
Get insights based on your data: ask AI to build a query for your database or upload a file you want to analyze.
Press Ctrl+Enter to submit your query, press
Select AI profile
Select model
Files
Name
DataSource
Bookmarks
Dashboards
Diagrams
Scripts
General
WIB
en
Writable
Smart Insert
36 : 16 : 1097
Sel: 0

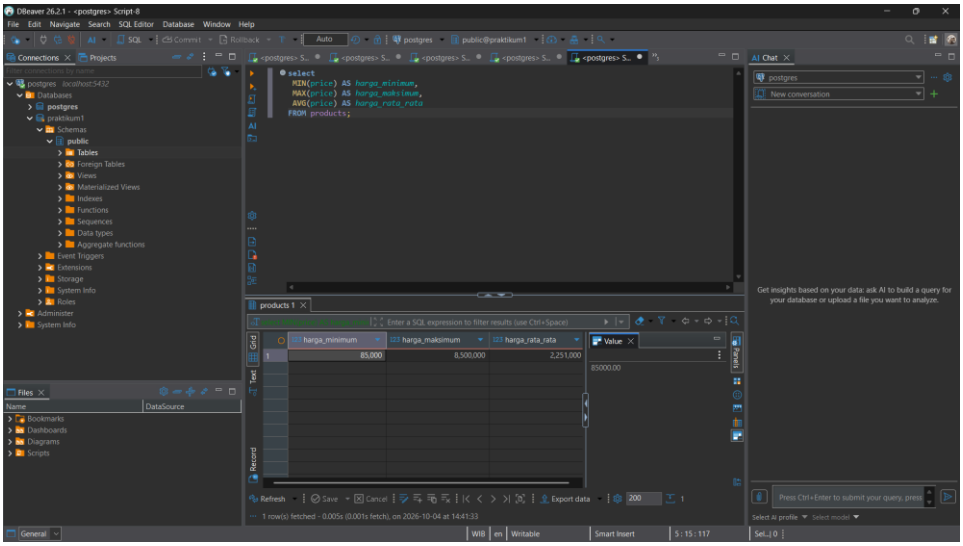

DBeaver 26.2.1 - <postgres> Script-7
(11, '2023-07-11', 850000),
(12, '2023-07-12', 2700000),
(13, '2023-07-13', 650000),
(14, '2023-07-14', 2300000),
(15, '2023-07-15', 8500000),
(16, '2023-07-16', 450000),
(17, '2023-07-17', 2900000),
(18, '2023-07-18', 650000),
(19, '2023-07-19', 1500000),
(20, '2023-07-20', 3000000),
(21, '2023-07-21', 850000),
(22, '2023-07-22', 2500000),
(23, '2023-07-23', 6500000),
(24, '2023-07-24', 550000),
(25, '2023-07-25', 7800000),
(26, '2023-07-26', 1700000),
(27, '2023-07-27', 900000),
(28, '2023-07-28', 2800000),
(29, '2023-07-29', 1250000),
(30, '2023-07-30', 350000);
Statistics 1
Name
Value
Updated Rows
60
Execute time
0.016s
Start time
Sun Oct 04 14:39:25 WIB 2026
Finish time
Sun Oct 04 14:39:25 WIB 2026
Query
INSERT INTO orders
(customer_id, order_date, total_amount)
VALUES
(1, '2023-06-01', 8500000),
(2, '2023-06-02', 2500000),
(3, '2023-06-03', 7800000),
(4, '2023-06-04', 650000),
(5, '2023-06-05', 2900000),
(6, '2023-06-06', 1700000),
(7, '2023-06-07', 450000),
(8, '2023-06-08', 6500000),
(9, '2023-06-09', 2300000),
(10, '2023-06-10', 850000),
AI Chat
postgres
New conversation
Get insights based on your data: ask AI to build a query for your database or upload a file you want to analyze.
Press Ctrl+Enter to submit your query, press
Select AI profile
Select model
Files
Name
DataSource
Bookmarks
Dashboards
Diagrams
Scripts
General
WIB
en
Writable
Smart Insert
64 : 1 : 1829
Sel: 0


DBeaver 26.2.1 - <postgres> Script-7
File Edit Navigate Search SQL Editor Database Window Help
(13, '2023-07-13', 650000),
(14, '2023-07-14', 2300000),
(15, '2023-07-15', 8500000),
(16, '2023-07-16', 450000),
(17, '2023-07-17', 2900000),
(18, '2023-07-18', 650000),
(19, '2023-07-19', 1500000),
(20, '2023-07-20', 3000000),
(21, '2023-07-21', 850000),
(22, '2023-07-22', 2500000),
(23, '2023-07-23', 6500000),
(24, '2023-07-24', 550000),
(25, '2023-07-25', 7800000),
(26, '2023-07-26', 1700000),
(27, '2023-07-27', 900000),
(28, '2023-07-28', 2800000),
(29, '2023-07-29', 1250000),
(30, '2023-07-30', 350000);
SELECT COUNT(*) AS jumlah_orders
FROM orders;
jumlah_orders
60
1 row(s) fetched - 0.001s, on 2026-10-04 at 14:39:40
Get insights based on your data: ask AI to build a query for your database or upload a file you want to analyze.

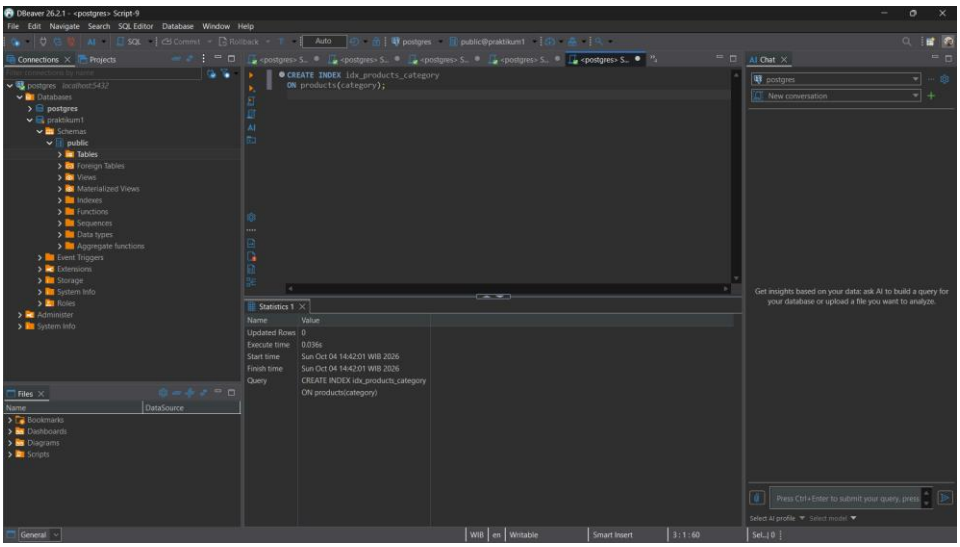

DBeaver 26.2.1 - <postgres> Script-7
File Edit Navigate Search SQL Editor Database Window Help
(16, '2023-07-16', 450000),
(17, '2023-07-17', 2900000),
(18, '2023-07-18', 650000),
(19, '2023-07-19', 1500000),
(20, '2023-07-20', 3000000),
(21, '2023-07-21', 850000),
(22, '2023-07-22', 2500000),
(23, '2023-07-23', 6500000),
(24, '2023-07-24', 550000),
(25, '2023-07-25', 7800000),
(26, '2023-07-26', 1700000),
(27, '2023-07-27', 900000),
(28, '2023-07-28', 2800000),
(29, '2023-07-29', 1250000),
(30, '2023-07-30', 350000);
SELECT COUNT(*) AS jumlah_orders
FROM orders;
SELECT 'products' AS tabel, COUNT(*) AS jumlah
FROM products
tabel
jumlah
products
40
1 row(s) fetched - 0.007s (0.001s fetch), on 2026-10-04 at 14:39:59
Get insights based on your data: ask AI to build a query for your database or upload a file you want to analyze.

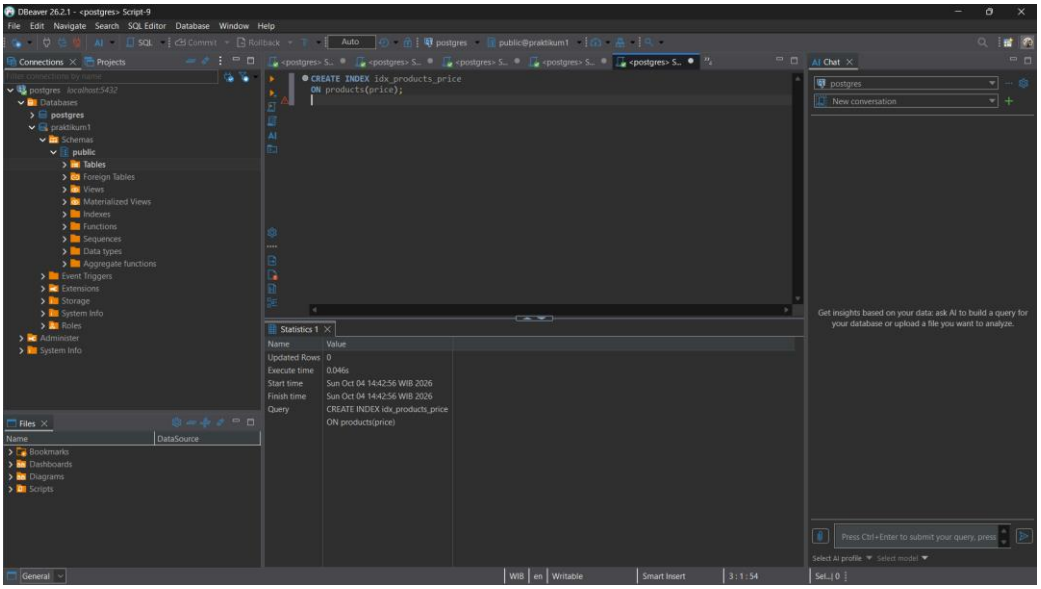

DBeaver 26.2.1 - <postgres> Script-7
File Edit Navigate Search SQL Editor Database Window Help
(20, '2023-07-20', 3000000),
(21, '2023-07-21', 850000),
(22, '2023-07-22', 2500000),
(23, '2023-07-23', 6500000),
(24, '2023-07-24', 550000),
(25, '2023-07-25', 7800000),
(26, '2023-07-26', 1700000),
(27, '2023-07-27', 900000),
(28, '2023-07-28', 2800000),
(29, '2023-07-29', 1250000),
(30, '2023-07-30', 350000);
SELECT COUNT(*) AS jumlah_orders
FROM orders;
SELECT 'products' AS tabel, COUNT(*) AS jumlah
FROM products
SELECT 'customers', COUNT(*)
FROM customers
?column?
count
customers
30
1 row(s) fetched - 0.001s, on 2026-10-04 at 14:40:46
Get insights based on your data: ask AI to build a query for your database or upload a file you want to analyze.

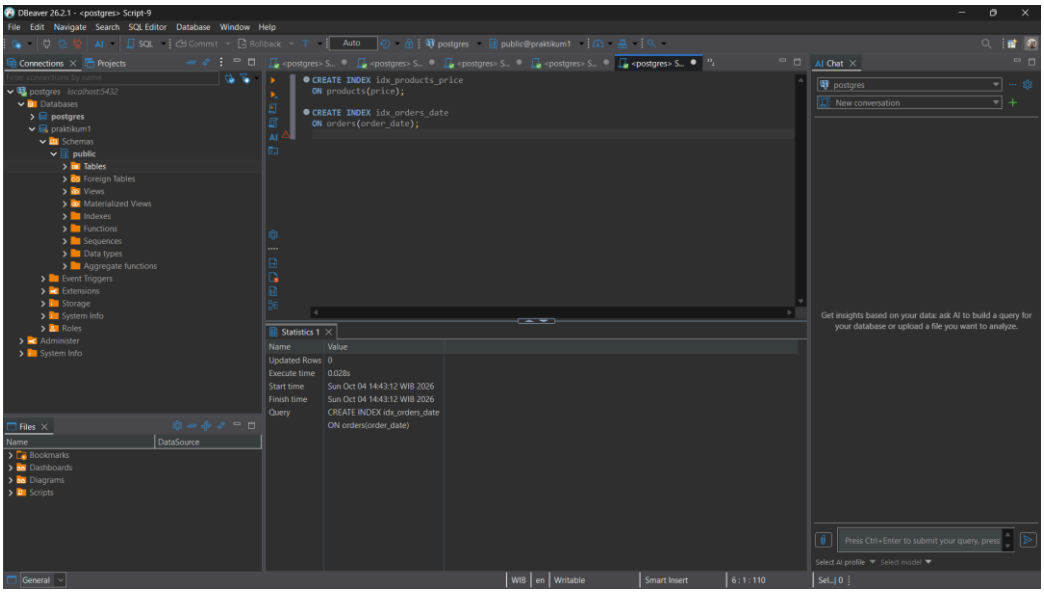


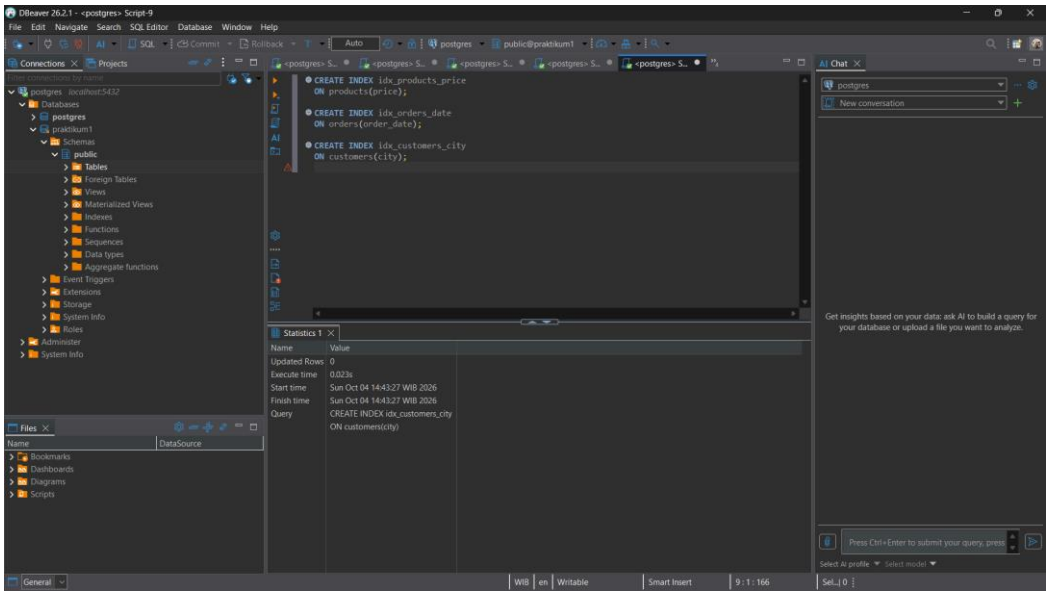


4. Sekarang kita lanjut ke tahap pembuatan index Products, Costumers dan Indexs.

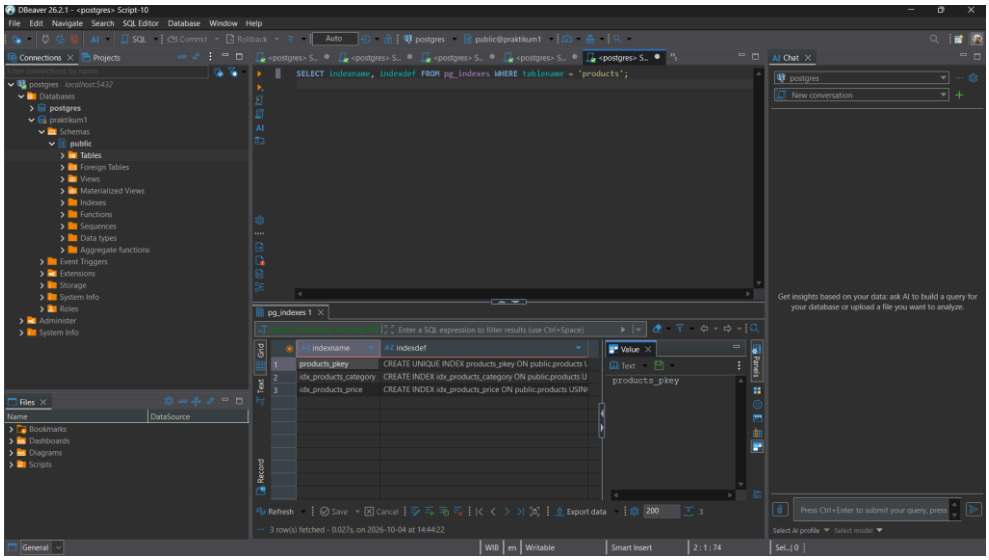


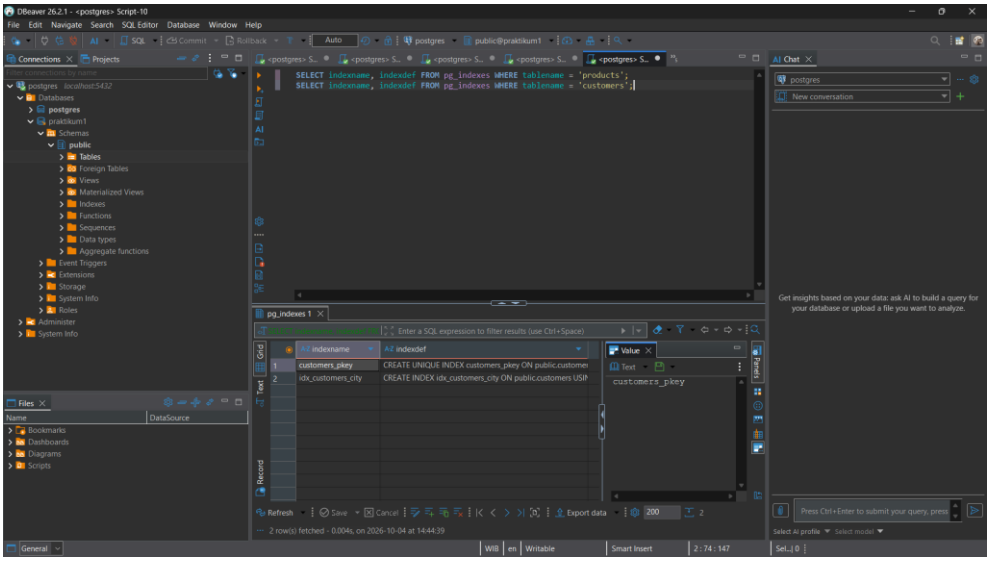

DBeaver 26.2.1 - <postgres> Script-9
CREATE INDEX idx_products_price
ON products(price);
Statistics 1
Name
Value
Updated Rows
0
Execute time
0.046s
Start time
Sun Oct 04 14:42:56 WIB 2026
Finish time
Sun Oct 04 14:42:56 WIB 2026
Query
CREATE INDEX idx_products_price
ON products(price)
AI Chat
Get insights based on your data: ask AI to build a query for your database or upload a file you want to analyze.

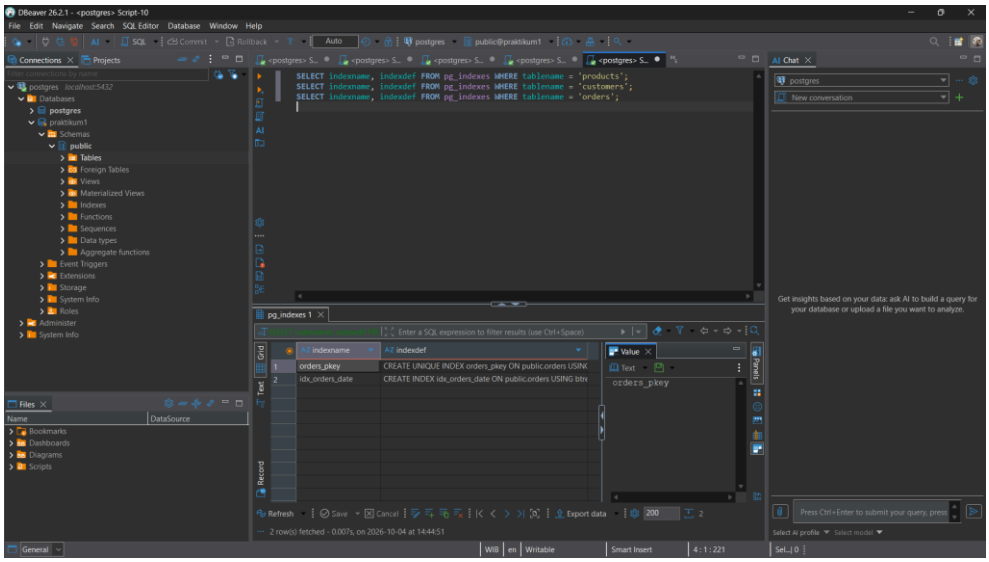

DBeaver 26.2.1 - <postgres> Script-9
CREATE INDEX idx_products_price
ON products(price);
CREATE INDEX idx_orders_date
ON orders(order_date);
Statistics 1
Name
Value
Updated Rows
0
Execute time
0.028s
Start time
Sun Oct 04 14:43:12 WIB 2026
Finish time
Sun Oct 04 14:43:12 WIB 2026
Query
CREATE INDEX idx_orders_date
ON orders(order_date)
AI Chat
Get insights based on your data: ask AI to build a query for your database or upload a file you want to analyze.

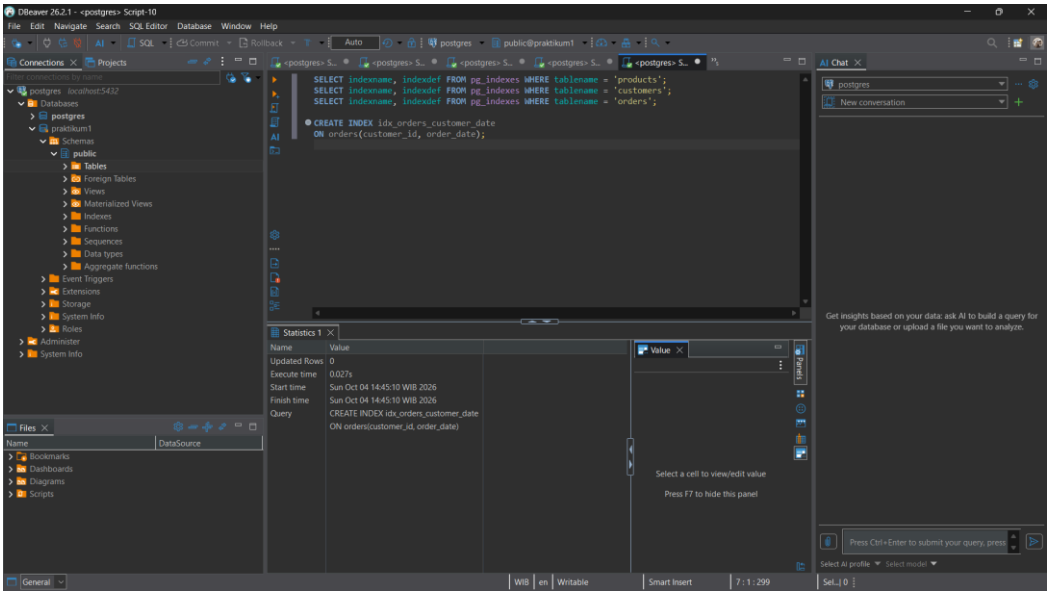

DBeaver 26.2.1 - <postgres> Script-9
CREATE INDEX idx_products_price
ON products(price);
CREATE INDEX idx_orders_date
ON orders(order_date);
CREATE INDEX idx_customers_city
ON customers(city);
Statistics 1
Name
Value
Updated Rows
0
Execute time
0.023s
Start time
Sun Oct 04 14:43:27 WIB 2026
Finish time
Sun Oct 04 14:43:27 WIB 2026
Query
CREATE INDEX idx_customers_city
ON customers(city)
AI Chat
Get insights based on your data: ask AI to build a query for your database or upload a file you want to analyze.

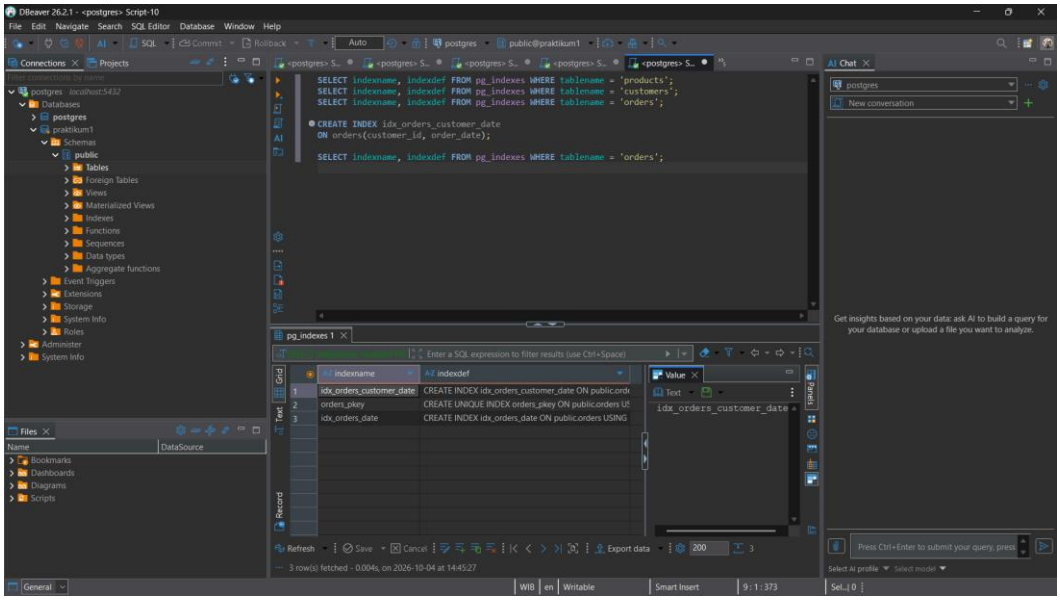
5. Oke, setelah pembuatan indexs, lanjut ke pemeriksaan Indexs.

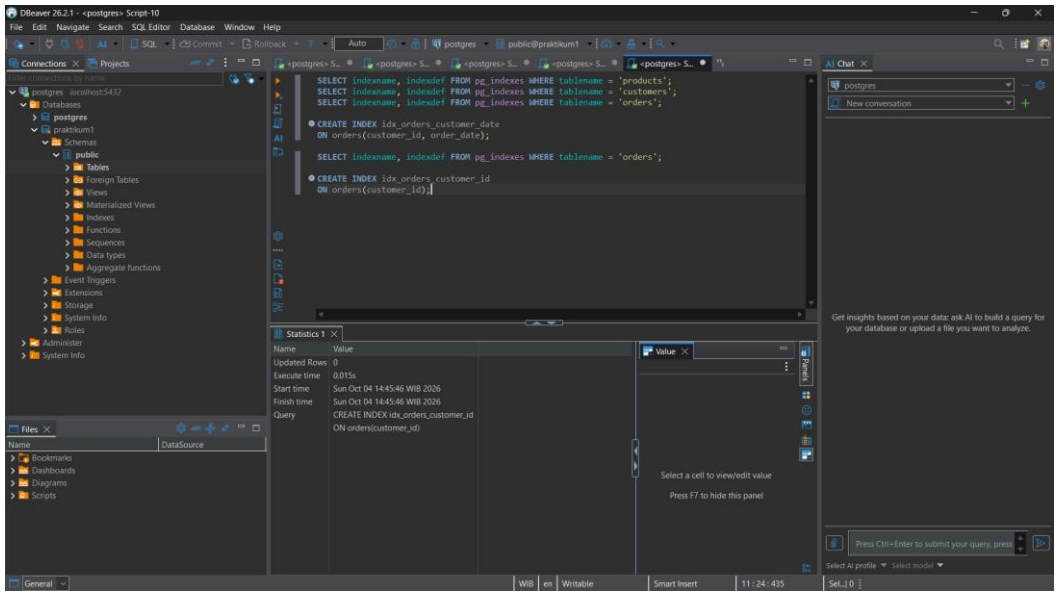


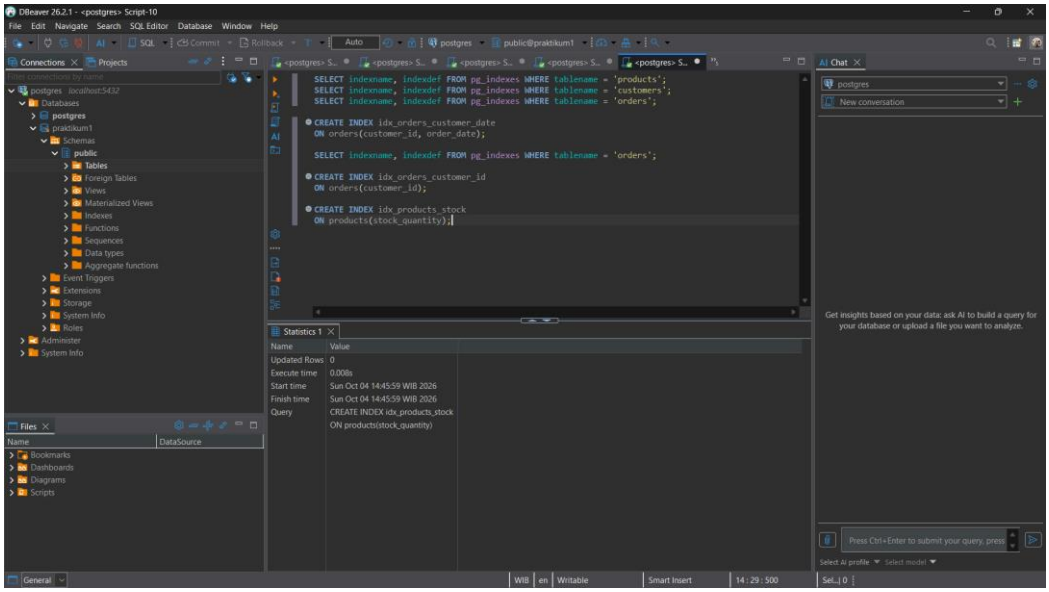





DBeaver 26.2.1 - <postgres> Script-10
SELECT indexname, indexdef FROM pg_indexes WHERE tablename = 'products';
SELECT indexname, indexdef FROM pg_indexes WHERE tablename = 'customers';
SELECT indexname, indexdef FROM pg_indexes WHERE tablename = 'orders';
CREATE INDEX idx_orders_customer_date
ON orders(customer_id, order_date);
Statistics 1
Name
Value
Updated Rows
0
Execute time
0.027s
Start time
Sun Oct 04 14:45:10 WIB 2026
Finish time
Sun Oct 04 14:45:10 WIB 2026
Query
CREATE INDEX idx_orders_customer_date
ON orders(customer_id, order_date)
Select a cell to view/edit value
Press F7 to hide this panel
AI Chat
Get insights based on your data: ask AI to build a query for your database or upload a file you want to analyze.


DBeaver 26.2.1 - <postgres> Script-10
SELECT indexname, indexdef FROM pg_indexes WHERE tablename = 'products';
SELECT indexname, indexdef FROM pg_indexes WHERE tablename = 'customers';
SELECT indexname, indexdef FROM pg_indexes WHERE tablename = 'orders';
CREATE INDEX idx_orders_customer_date
ON orders(customer_id, order_date);
SELECT indexname, indexdef FROM pg_indexes WHERE tablename = 'orders';
pg_indexes 1
indexname
indexdef
idx_orders_customer_date
CREATE INDEX idx_orders_customer_date ON public.orde
orders_pkey
CREATE UNIQUE INDEX orders_pkey ON public.orders US
idx_orders_date
CREATE INDEX idx_orders_date ON public.orders USING
3 row(s) fetched - 0.004s, on 2026-10-04 at 14:45:27
AI Chat
Get insights based on your data: ask AI to build a query for your database or upload a file you want to analyze.


DBeaver 26.2.1 - <postgres> Script-10
SELECT indexname, indexdef FROM pg_indexes WHERE tablename = 'products';
SELECT indexname, indexdef FROM pg_indexes WHERE tablename = 'customers';
SELECT indexname, indexdef FROM pg_indexes WHERE tablename = 'orders';
CREATE INDEX idx_orders_customer_date
ON orders(customer_id, order_date);
SELECT indexname, indexdef FROM pg_indexes WHERE tablename = 'orders';
CREATE INDEX idx_orders_customer_id
ON orders(customer_id);
Statistics 1
Name
Value
Updated Rows
0
Execute time
0.015s
Start time
Sun Oct 04 14:45:46 WIB 2026
Finish time
Sun Oct 04 14:45:46 WIB 2026
Query
CREATE INDEX idx_orders_customer_id
ON orders(customer_id)
Select a cell to view/edit value
Press F7 to hide this panel
AI Chat
Get insights based on your data: ask AI to build a query for your database or upload a file you want to analyze.



Jawaban Pertanyaan Diskusi

1. **Apa fungsi index pada basis data ?**

   Index berfungsi untuk membantu DBMS menemukan data tertentu secara lebih efisien tanpa harus membaca seluruh baris dalam tabel. Index dapat mempercepat proses pencarian data menggunakan kondisi WHERE, operasi JOIN, serta operasi tertentu seperti ORDER BY dan GROUP BY, bergantung pada struktur index yang digunakan.

2. **Apakah setiap kolom harus diberikan index ?**

   Tidak, karena tidak semua kolom sering digunakan untuk pencarian atau pengolahan data. Index sebaiknya dibuat pada kolom yang sering digunakan dalam query, seperti kolom yang digunakan pada kondisi WHERE atau JOIN. Pembuatan index pada setiap kolom justru dapat menambah kebutuhan penyimpanan dan biaya pemeliharaan database.

3. **Mengapa index dapat memperbesar biaya INSERT, UPDATE dan DELETE ?**

   Karena setiap kali data ditambahkan, diubah, atau dihapus, PostgreSQL juga perlu memperbarui struktur index yang berkaitan dengan data tersebut. Proses pemeliharaan ini membutuhkan sumber daya dan waktu tambahan, sehingga semakin banyak index yang dimiliki suatu tabel, semakin besar pula biaya yang diperlukan untuk operasi perubahan data.

4. **Apa perbedaan single-column index dan composite index ?**

   Single-column index adalah index yang dibuat pada satu kolom dalam tabel, sedangkan composite index adalah index yang dibuat menggunakan dua kolom atau lebih.

5. **Mengapa urutan kolom pada composite index penting ?**

   Urutan kolom pada composite index menentukan pola pencarian yang dapat didukung oleh index tersebut. Misalnya, index (customer_id, order_date) lebih sesuai untuk query yang mencari berdasarkan customer_id, atau berdasarkan customer_id sekaligus rentang order_date.

Susunan (order_date, customer_id) tidak identik karena urutan kolom awalnya berbeda, sehingga pemanfaatannya untuk query juga dapat berbeda.

6. **Bagaimana index dapat membantu operasi JOIN ?**

   Index dapat membantu operasi JOIN dengan mempercepat pencarian baris yang memiliki nilai kolom penghubung yang sesuai pada tabel lain. Contohnya, pada tabel orders terdapat kolom customer_id sebagai foreign key yang menghubungkan data pesanan dengan tabel customers. Index pada kolom yang digunakan untuk menghubungkan kedua tabel dapat membantu PostgreSQL menemukan data yang cocok dengan lebih efisien. Namun, PostgreSQL tidak otomatis membuat index pada kolom foreign key.

7. **Apa risiko membuat terlalu banyak index pada suatu table ?**

   Terlalu banyak index dapat meningkatkan penggunaan ruang penyimpanan dan biaya pemeliharaan database. Setiap operasi INSERT, UPDATE, dan DELETE juga dapat membutuhkan proses tambahan untuk memperbarui index yang terkait. Oleh karena itu, index perlu dibuat sesuai kebutuhan query dan tidak berlebihan agar pengelolaan database tetap efisien.

Refleksi Praktikum

1. **Index mana saja yang berhasil dibuat dan pada kolom apa ?**

   Index yang dibuat pada database toko material terdiri dari dua *single-column index* dan satu *composite index*. *Single-column index* yang dibuat adalah idx_products_category pada kolom category dan idx_products_price pada kolom price di tabel products. Sementara itu, *composite index* yang dibuat adalah idx_orders_customer_date pada kolom customer_id dan order_date di tabel orders. Index tersebut dibuat untuk mendukung pencarian data berdasarkan kategori produk, harga, serta pelanggan dan tanggal pesanan.

2. **Apa konsekuensi jika sebuah tabel memiliki terlalu banyak index ?**

   Jika sebuah tabel memiliki terlalu banyak index, kebutuhan ruang penyimpanan database akan meningkat karena setiap index membutuhkan ruang tersendiri. Selain itu, operasi INSERT, UPDATE, dan DELETE dapat menjadi lebih lambat karena PostgreSQL harus memperbarui index yang berkaitan dengan perubahan data. Oleh karena itu, jumlah index perlu disesuaikan dengan kebutuhan agar tidak menimbulkan biaya pemeliharaan yang berlebihan.

3. **Bagaimana menentukan apakah sebuah kolom layak diberikan index ?**

   Kolom layak diberikan index apabila sering digunakan dalam query, terutama pada kondisi WHERE, operasi JOIN, atau pengurutan data menggunakan ORDER BY. Pemilihan index juga perlu mempertimbangkan pola pencarian dan kebutuhan aplikasi. Pada database toko material, kolom category dan price diberikan index karena digunakan untuk pencarian produk berdasarkan kategori dan rentang harga. Dengan demikian, index sebaiknya dibuat berdasarkan kebutuhan query dan bukan pada semua kolom tabel.

Kesimpulan Praktikum

Praktikum ini membuktikan keterkaitan langsung antara konfigurasi basis data dan indexing, Konfigurasi indexing bukanlah proses menambahkan indeks ke sebanyak mungkin kolom melainkan penyesuaian terukur berdasarkan pola akses data. Index Primary Key terbuat secara otomatis oleh PostgreSQL sedangkan kolom Foreign Key dan kolom filter harus dikonfigurasi secara eksplisit agar pengolahan relasi dan filter query berjalan optimal tanpa menimbulkan overhead berlebih.